

SplitEvent - Dossier de presentation

Table des matières

Explication	3
Les objectifs fonctionnels du projet :	3
Authentification et profil utilisateur	4
Création et gestion des événements :	5
Invitations et calendrier :	5
Amis, présence et carnet d'invités :	6
Dépenses partagées et calcul des soldes :	7
Base de données Supabase :	7
Sécurité et Row Level Security :	8
Sécurité et Row Level Security :	9
Clean Architecture mise en place :	9
Améliorations possibles :	10
Conclusion :	11

Explication

SplitEvent est une application mobile construite avec Expo et React Native.

Son objectif est de simplifier l'organisation d'événements entre amis, depuis la création de l'événement jusqu'au suivi des dépenses et des remboursements.

Le projet répond à un besoin concret : réunir dans une seule interface les invitations, le calendrier, les participants, les dépenses partagées, les soldes et l'historique des remboursements.

L'application utilise Supabase comme backend pour l'authentification, la base de données, les règles de sécurité et certaines fonctions temps réel comme la présence en ligne des amis.

Le code a été refactorisé vers une clean Architecture stricte afin de séparer clairement le domaine métier, les cas d'utilisation, l'infrastructure technique et la présentation.

Cette séparation permet de conserver le fonctionnement actuel tout en rendant le projet plus maintenable, plus testable et plus simple à faire évoluer.

Le dossier présente les objectifs, les fonctionnalités, la base de données, les choix techniques, l'architecture et les améliorations possibles.

Les objectifs fonctionnels du projet :

Le premier objectif est de permettre à un utilisateur de créer un compte, de se connecter et de gérer son profil. Le profil contient le nom affiché dans l'application pour les autres contacts ainsi qu'un numéro de compte bancaire utile pour les remboursements.

Le deuxième objectif est de créer un événement. L'onglet de création est volontairement limité à cette responsabilité : saisir le titre, la date, l'heure, le lieu, la description et les amis à inviter.

Le troisième objectif est de gérer les invitations. Un utilisateur qui reçoit une invitation voit une notification, il peut accepter ou refuser l'événements => les événements acceptés apparaissent dans le calendrier.

Le quatrième objectif est de suivre les dépenses d'un événement. Chaque dépense indique qui à payer, pour qui ainsi que le montant et la date.

Le cinquième objectif est de calculer automatiquement les soldes. L'application détermine qui doit rembourser qui, puis permet de marquer un remboursement comme effectué.

Enfin le dernier objectif est de conserver un historique des remboursements afin de ne pas perdre la trace des paiements déjà réalisés.

Authentification et profil utilisateur

L'authentification est gérée par Supabase Auth. L'utilisateur peut créer un compte avec un nom, un email et un mot de passe. Après connexion, l'application récupère l'utilisateur courant et conserve ses informations dans le contexte de présentation.

Le nom utilisateur est stocké dans les métadonnées Supabase Auth. Le numéro de compte bancaire est également stocké dans ces métadonnées. Ce choix évite de créer une table supplémentaire uniquement pour deux champs simples liés au compte connecté.

La page Profil permet de modifier le nom et le numéro de compte bancaire. Le champ email est affiché mais non éditable, car il identifie le compte d'authentification.

Dans la Clean Architecture, les écrans ne modifient pas directement Supabase. Ils utilisent le contexte de présentation, qui appelle les cas d'utilisation, qui eux-mêmes passent par un repository d'authentification.

Cette organisation isole le choix Supabase. Si un autre fournisseur d'authentification était utilisé plus tard, il suffirait de remplacer l'implémentation du repository sans modifier les ecrans.

Création et gestion des événements :

La création d'évènement se fait dans l'onglet Creer. Cette page ne sert plus à afficher tous les évènements : elle a une seule responsabilité, ce qui rend l'interface plus claire.

Le formulaire contient un titre, une date au format JJ/MM/AAAA, une heure, un lieu, une description et une liste d'amis à inviter. La saisie de date ajoute automatiquement les séparateurs et refuse les dates invalides.

Les amis à inviter sont affichés sous forme de liste verticale avec le nom, l'email et une case de sélection. Cela rend la sélection plus lisible qu'une simple suite de pastilles.

Lors de la validation, l'application crée une ligne dans la table événements puis crée les invitations associées dans la table invitations.

Les événements créés ou acceptés ne sont plus affichés dans l'onglet de création. Ils apparaissent dans le calendrier, ce qui sépare création et consultation.

La suppression d'un évènement reste possible pour son créateur depuis le calendrier. La base utilise une suppression en cascade pour retirer les données liées.

Invitations et calendrier :

Les invitations sont stockées avec un statut : pending, accepted ou declined. Une invitation reçue reste visible comme notification tant qu'elle est en attente.

Quand l'utilisateur accepte une invitation, l'évènement devient visible dans le calendrier.

Si l'invitation est refusée, l'évènement ne s'ajoute pas au calendrier.

Le calendrier affiche les évènements créés par l'utilisateur et ceux qu'il a acceptés. Il trie les évènements à partir de la date au format JJ/MM/AAAA et de l'heure.

Chaque carte du calendrier affiche le jour, le mois, le titre, l'heure, le lieu, la description et les participants acceptés.

Le créateur d'un évènement peut aussi inviter un ami oublié directement depuis le calendrier. Cette action crée une nouvelle invitation sans recréer l'évènement.

Le calendrier sert donc de centre opérationnel pour les évènements existants, tandis que l'onglet Créer reste uniquement dédié à la création.

Amis, présence et carnet d'invités :

La fonctionnalité Mes amis permet d'ajouter des contacts. Une relation d'ami contient un nom, un email, le nom du demandeur, l'email du demandeur, un statut et l'identifiant du créateur.

Le statut d'une relation peut être invited, accepted ou declined. Cela permet de différencier une demande envoyée, une demande acceptée et une demande refusée.

Le carnet d'amis sert ensuite à inviter plus rapidement des personnes à un événement. Seuls les amis acceptés sont proposés dans la liste d'invitation.

Une fonctionnalité de présence en ligne existe aussi. Elle utilise les canaux Supabase Realtime pour signaler les utilisateurs connectés.

Dans la Clean Architecture, cette présence est isolée dans l'infrastructure realtime. Le hook React de présentation ne connaît pas le détail Supabase: il se contente de s'abonner à une source de présence.

Dépenses partagées et calcul des soldes :

Une dépense appartient à un évènement. Elle contient un titre, un montant, la personne qui a payé, son email, une date et le créateur de la ligne.

Les participants concernés par la dépense sont stockés dans une table séparée `expense_participants`. Ce choix permet de représenter une dépense partagée entre plusieurs personnes sans dupliquer la dépense principale.

Le calcul des soldes est une vraie règle métier. Il a donc été placé dans le domaine, dans `balance-service`, sans dépendance à React ou Supabase.

La logique calcule combien chaque participant a payé, combien il devait payer, puis génère les lignes de remboursement minimales entre débiteurs et créditeurs.

Quand une personne confirme qu'un remboursement a été effectué, une ligne est ajoutée dans la table `repayments`. Les soldes restants sont recalculés en soustrayant les remboursements déjà confirmés.

L'historique des remboursements reste visible afin de conserver une trace claire des paiements effectués.

Base de données Supabase :

La base de données est définie dans `supabase/schema.sql`. Le schéma crée les tables `events`, `friends`, `invitations`, `expenses`, `expense_participants` et `repayments`.

La table `events` stocke les informations principales d'un évènement: titre, date, heure, lieu, description, créateur, email du créateur et date de création.

La table `invitations` relie un évènement à une personne invitée. Elle porte le statut de l'invitation, ce qui permet de piloter les notifications et l'apparition dans le calendrier.

La table friends gère le carnet d'amis. Elle stocke à la fois la personne invitée et les informations du demandeur, car un utilisateur peut recevoir une demande d'une autre personne.

La table expenses stocke les dépenses principales. La table expense_participants stocke les participants de chaque dépense, ce qui correspond à une relation un-a-plusieurs.

La table repayments stocke les remboursements confirmés. Elle permet d'éviter qu'une dette payée réapparaisse après rechargement de l'application.

Les contraintes de clé étrangère assurent la cohérence. Les suppressions en cascade évitent de garder des invitations, dépenses ou remboursements orphelins après suppression d'un événement.

Securité et Row Level Security :

Supabase permet d'activer Row Level Security sur chaque table. Le projet utilise ces politiques pour limiter la lecture et l'écriture aux utilisateurs concernés.

Un utilisateur peut lire ses propres événements et les événements auxquels il est invité. Il peut créer un événement uniquement en son nom.

Les invitations peuvent être lues par le créateur de l'événement ou par la personne invitée. Cela évite qu'un utilisateur voie des invitations qui ne le concernent pas.

Les dépenses sont accessibles uniquement aux personnes qui peuvent contribuer à l'événement. Cette règle est importante, car une dépense contient des informations financières.

Les remboursements suivent la même logique : seules les personnes liées à l'événement peuvent les lire ou les créer.

Ce modèle de sécurité est plus robuste qu'une vérification uniquement côté application. Même si un client malveillant tente une requête directe, la base applique ses propres restrictions.

Securité et Row Level Security :

SQLite est une excellente base locale. Elle est simple, rapide et très adaptée à une application hors ligne ou à un prototype sans synchronisation entre utilisateurs.

SplitEvent a cependant besoin d'un backend partage. Les événements, invitations, amis, dépenses et remboursements doivent être visibles par plusieurs utilisateurs sur plusieurs appareils.

Avec SQLite seul, il faudrait construire un serveur, une API, une authentification, une synchronisation et un système de sécurité. Cela ajouterait beaucoup de complexité au projet.

Supabase fournit directement une base PostgreSQL, l'authentification, les politiques RLS, les canaux temps réel et un SDK JavaScript compatible avec Expo.

Le choix Supabase est donc cohérent avec les besoins collaboratifs de SplitEvent. Il permet de se concentrer sur les fonctionnalités métier plutôt que sur la création d'une infrastructure complète.

SQLite pourrait rester utile plus tard pour un cache hors ligne. Mais comme source de vérité principale, Supabase est plus adapté à une application multi-utilisateur connectée.

Clean Architecture mise en place :

Le projet a été refactorisé vers une Clean Architecture stricte. L'objectif est que les règles métier ne dépendent ni de React Native, ni de Supabase, ni d'Expo.

Le dossier src/domain contient les entités et les interfaces de repositories. Il contient aussi la logique pure de calcul des soldes et de gestion du format de date.

Le dossier src/application contient les cas d'utilisation. Ces fonctions orchestrent les opérations mais ne connaissent pas les détails techniques de Supabase.

Le dossier src/infrastructure contient les implémentations concrètes: repositories Supabase, mappers entre lignes de base et entites, présence temps réel.

Le dossier src/présentation contient les contextes React et les hooks. Les écrans Expo utilisent ces contextes pour déclencher les actions.

Cette architecture inverse les dépendances: le domaine définit ce dont il a besoin, et l'infrastructure vient implémenter ces contrats.

Le résultat est plus testable. On peut tester balance-service sans base de données, ou remplacer un repository Supabase par un faux repository dans un test.

Améliorations possibles :

Une première amélioration serait d'ajouter des tests unitaires sur le domaine: calcul des soldes, validation des dates, calcul des remboursements restants.

Une deuxième amélioration serait de créer une vraie table profiles si le profil utilisateur devient plus riche: avatar, téléphone, préférences, devise, langue ou historique bancaire.

Une troisième amélioration serait d'ajouter un mode hors ligne avec cache local. SQLite pourrait alors être utilisé comme cache synchronisé avec Supabase.

Une quatrième amélioration serait de renforcer les formulaires avec des composants réutilisables : champ de date, champ IBAN, sélecteur d'amis, champ montant.

Une cinquième amélioration serait de permettre plusieurs devises et une conversion automatique, ce qui demanderait une nouvelle règle métier dans le domaine.

Une sixième amélioration serait de rendre les remboursements plus précis : remboursement partiel, annulation d'un remboursement, commentaire ou preuve de paiement.

Une septième amélioration serait d'ajouter des tests end-to-end pour vérifier le parcours complet : inscription, création d'évènement, invitation, dépense, remboursement.

Une huitième amélioration serait d'ajouter un chat en ligne.

Enfin, la partie design pourrait être enrichie avec un vrai calendrier mensuel, une meilleure visualisation des participants et des notifications push.

Conclusion :

SplitEvent est une application collaborative de gestion d'évènements et de dépenses partagées. Elle couvre les fonctions essentielles : compte utilisateur, profil, amis, invitations, calendrier, dépenses, soldes et remboursements.

La base Supabase répond bien au besoin multi-utilisateur, car elle combine stockage partage, authentification, sécurité et temps réel.

La Clean Architecture rend maintenant le projet plus solide : les écrans restent concentrés sur l'interface, le domaine contient les règles métier, l'application orchestre les cas d'utilisation et l'infrastructure isole Supabase.

Le projet peut continuer à évoluer sans mélanger progressivement la logique métier, les composants React et les appels réseau.

Les prochaines évolutions devraient prioriser les tests, la validation avancée des formulaires, le cache hors ligne et une expérience calendrier plus complète.